Taller guiado · SmartLibrary

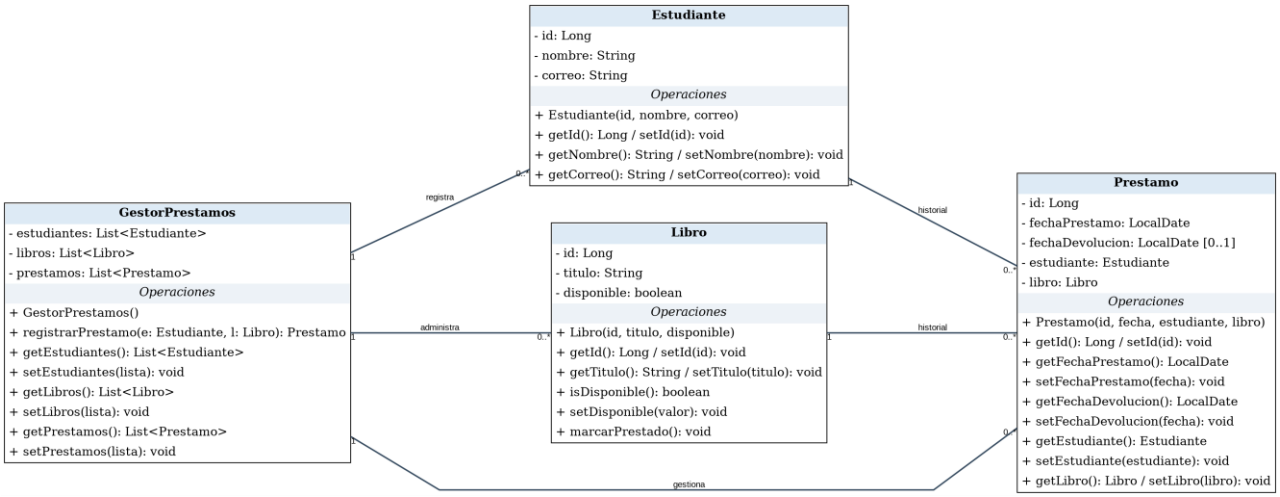
Por: Shary Velasquez

Actividad 2 · Asignar responsabilidades

Operación o decisión	Clase responsable	Justificación
Recibir POST /api/prestamos	PrestamoController	Recibe la solicitud HTTP y delega en el servicio; no debe contener reglas de negocio.
Buscar estudiante por identificador	EstudianteRepository, invocado por PrestamoService	El repositorio consulta por id; el servicio coordina el caso de uso y trata el resultado inexistente.
Buscar libro por identificador	LibroRepository, invocado por PrestamoService	Encapsula la consulta del libro. findByIdForUpdate obtiene el libro con bloqueo pesimista.
Validar disponibilidad	PrestamoService y Libro	El servicio verifica R1 y R5; Libro conoce su propio estado mediante isDisponible(). Se aplica Information Expert al estado del libro.
Crear el préstamo	PrestamoService	Coordina la creación y tiene estudiante, libro y fecha disponibles; aplica Creator al construir Prestamo.
Cambiar disponibilidad	Libro	marcarComoPrestado() cambia su propio atributo disponible a false, manteniendo alta cohesión.
Guardar cambios	PrestamoRepository y JPA dentro de la transacción	save(prestamo) persiste el préstamo. El cambio del Libro administrado se sincroniza por dirty checking al confirmar la transacción; no se invoca save(libro) explícitamente.

Devolver respuesta HTTP	PrestamoController y GlobalExceptionHandler	El Controller produce 201 Created y 200 OK; GlobalExceptionHandler traduce errores de validación y negocio a 400, 404 y 409.

Actividad 1. Lectura y observaciones del modelo inicial

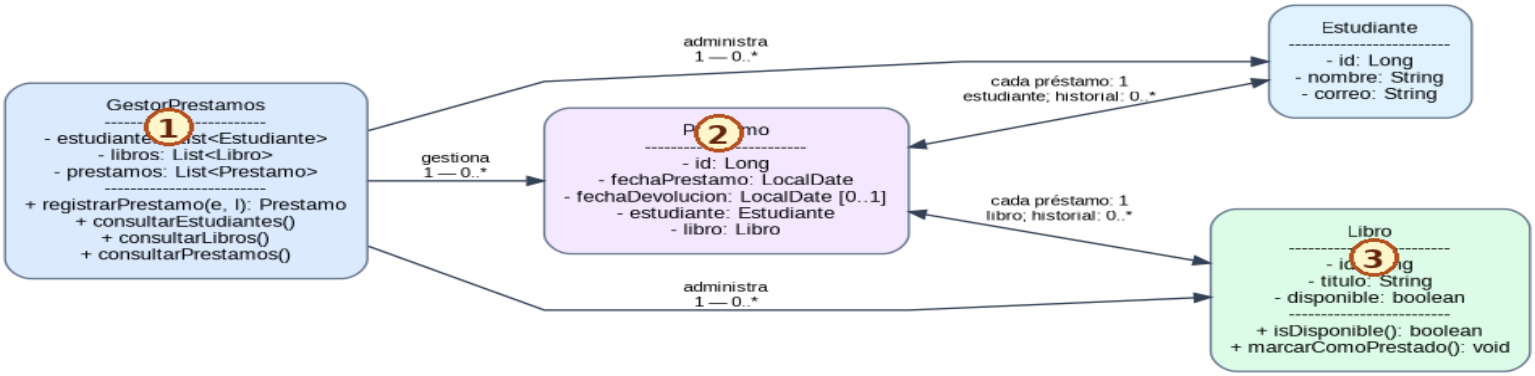


GestorPrestamos concentra la administración de las listas de estudiantes, libros y préstamos, así como la coordinación del registro y las consultas. Esta concentración reúne responsabilidades de distinta naturaleza y reduce la cohesión. En el diseño REST, la gestión HTTP, las reglas del caso de uso y la persistencia se distribuyen entre Controller, Service y Repository.

Estudiante representa id, nombre y correo. Libro contiene id, título y disponibilidad; por ello conoce su estado y ofrece isDisponible() y marcarComoPrestado(). Prestamo representa el registro histórico con fechaPrestamo, fechaDevolucion opcional y referencias obligatorias al estudiante y al libro.

Las asociaciones Estudiante–Prestamo y Libro–Prestamo son estructurales. Cada préstamo tiene exactamente un estudiante y un libro; cada estudiante y cada libro pueden aparecer en cero o muchos préstamos históricos. No se justifica composición ni agregación porque las entidades mantienen ciclos de vida independientes.

Figura 1. Modelo inicial conceptual de SmartLibrary



Observaciones del modelo inicial: (1) GestorPrestamos concentra listas, registro y consultas, por lo que presenta baja cohesión; (2) la clase gestora administra directamente entidades que deberían tener responsabilidades separadas; (3) Libro debe conservar la responsabilidad de consultar y modificar su disponibilidad; (4) Prestamo representa el vínculo histórico con exactamente un estudiante y un libro. Estas observaciones justifican la separación Controller - Service - Repository. Las asociaciones del dominio se mantienen sin agregación ni composición.

Actividad 2. Asignación de responsabilidades

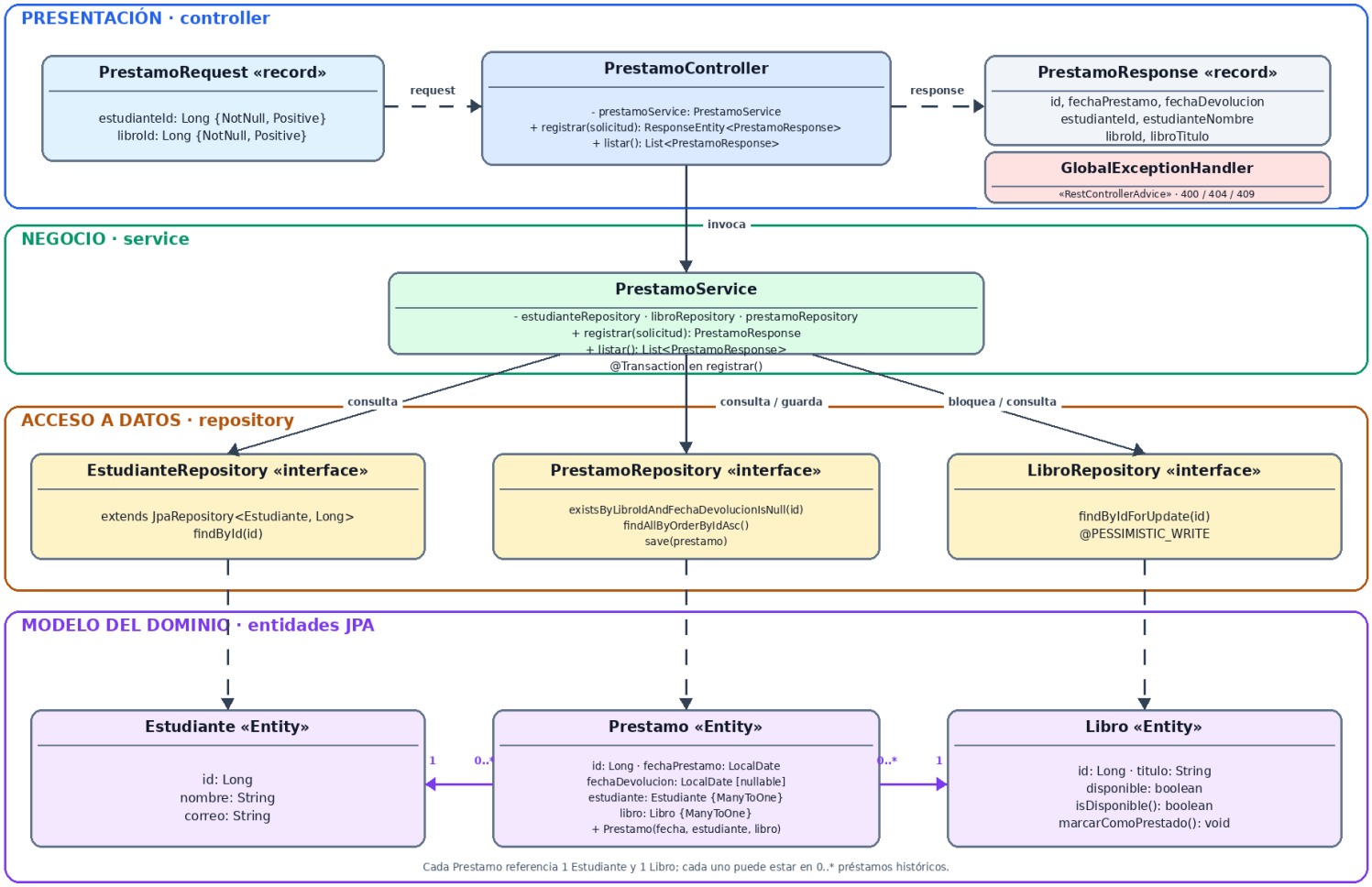
La asignación respeta la dirección Controller → Service → Repository. El servicio coordina, los repositorios consultan y persisten, y las entidades mantienen el estado que les corresponde.

Detalle de la implementación: LibroRepository.findByIdForUpdate usa PESSIMISTIC_WRITE. PrestamoService cambia la disponibilidad del Libro administrado por JPA y guarda Prestamo. El cambio del libro se persiste mediante el dirty checking de JPA al confirmar la transacción; el código no hace una llamada explícita a libroRepository.save().

Actividad 3. Diagrama de clases por capas

El diagrama refleja las clases y las operaciones del repositorio. PrestamoController mantiene una referencia a PrestamoService; PrestamoService mantiene referencias a EstudianteRepository, LibroRepository y PrestamoRepository. Las interfaces de repositorio extienden JpaRepository, y Spring Data JPA proporciona sus implementaciones en ejecución.

Figura 2. Diagrama de clases por capas — SmartLibrary



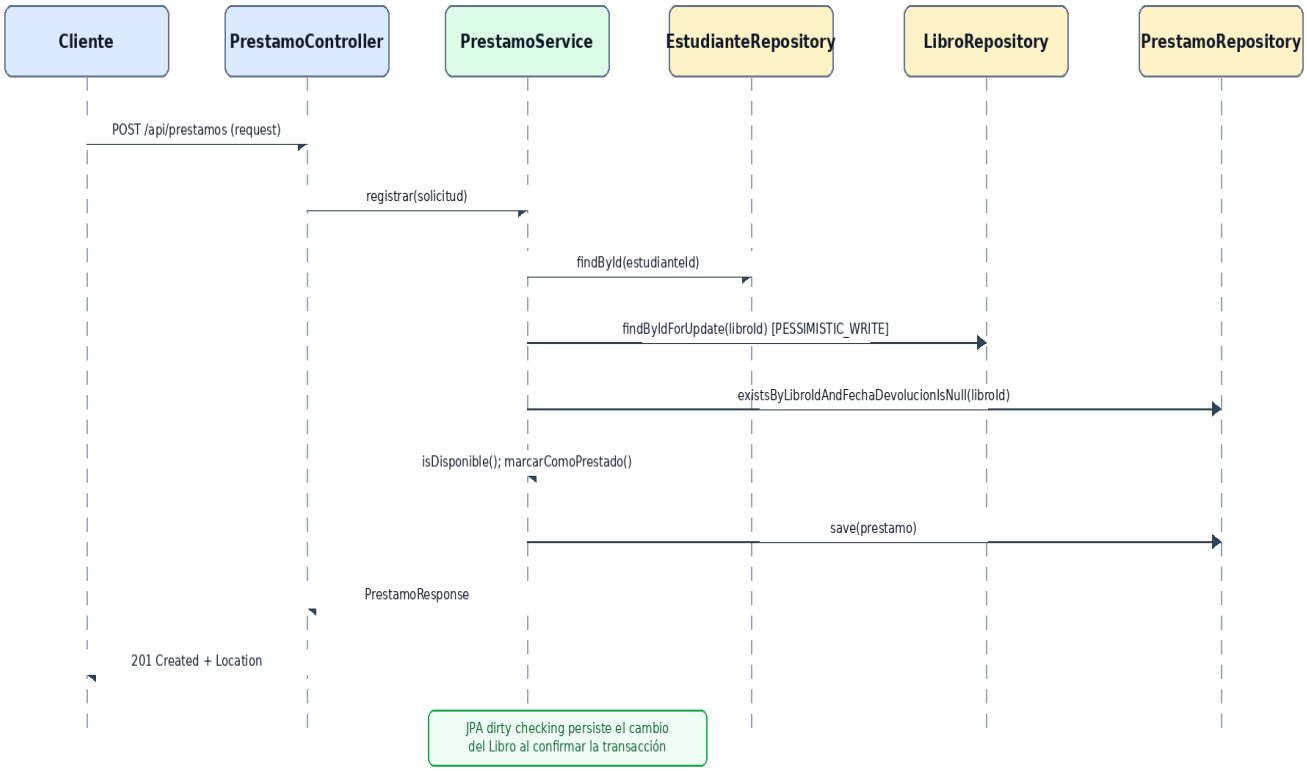
Multiplicidades del dominio: Prestamo se asocia con 1 Estudiante y 1 Libro; desde Estudiante y Libro pueden existir 0..* préstamos históricos. En el código, Prestamo mantiene las referencias con @ManyToOne(optional = false, fetch = LAZY), mientras que las entidades Estudiante y Libro no mantienen colecciones inversas.

Las dependencias entre capas se muestran como Controller - Service - Repository. PrestamoRequest y PrestamoResponse son DTOs: el primero valida los identificadores de entrada y el segundo evita exponer directamente entidades y proxies JPA. GlobalExceptionHandler centraliza las respuestas de error. No se incorporan herencia, composición ni agregación porque no hay relaciones de ese tipo en el modelo.

GRASP: PrestamoController cumple el rol de Controller GRASP porque recibe el evento del sistema (POST /api/prestamos) y delega su atención. PrestamoService coordina las reglas del caso de uso y la transacción; además construye Prestamo (Creator). Libro es Information Expert sobre su propio estado disponible. GlobalExceptionHandler centraliza errores de presentación.

Actividad 4. Diagrama de secuencia del préstamo

Figura 3. Diagrama de secuencia — POST /api/prestamos



el Controller recibe y valida PrestamoRequest; el Service busca al estudiante, obtiene el libro mediante findByIdForUpdate, comprueba que esté disponible y que no exista un préstamo activo, cambia el estado del libro, construye Prestamo con LocalDate.now(), guarda el préstamo y devuelve PrestamoResponse. El Controller responde 201 Created e incluye Location.

Alternativas: estudiante inexistente y libro inexistente producen 404; libro no disponible o con préstamo activo produce 409; solicitud inválida produce 400 antes de registrar el préstamo. La transacción permite revertir las modificaciones si la operación falla.

Actividad 5. Preparación e implementación Spring Boot

El repositorio utiliza Java 21, Spring Boot 3.5.7, Maven, Spring Web, Spring Data JPA, Spring Validation, H2 y JUnit 5. Los paquetes implementados son controller, service, repository, model, dto y exception.

El endpoint POST /api/prestamos recibe estudianteId y libroId. La fecha se genera en el servidor con LocalDate.now(). GET /api/prestamos consulta el historial.

PrestamoService.registrar es @Transactional; la consulta bloqueante del libro serializa intentos concurrentes. No existe una operación de devolución, por lo que fechaDevolucion permanece nula en los préstamos creados y estos continúan activos.

El controlador devuelve PrestamoResponse en lugar de entidades JPA. Esto evita exponer directamente las relaciones LAZY y proxies de persistencia en el contrato HTTP.

Actividad 6. Implementación y evidencias de pruebas

La evidencia publicada en docs/evidencias/pruebas-maven.txt registra la ejecución de PrestamoControllerIntegrationTest: 9 pruebas, 0 fallos, 0 errores y 0 omitidas; Maven informa BUILD SUCCESS. La evidencia de compilación separada registra BUILD SUCCESS, aunque en esa ejecución particular las pruebas aparecen omitidas; por eso se toma pruebas-maven.txt como evidencia de la suite ejecutada.

Escenario	Resultado observado	Verificación
Préstamo disponible	201 Created	Préstamo registrado y libro pasa a no disponible.
Libro no disponible	409 Conflict	No crea un préstamo nuevo.
Estudiante inexistente	404 Not Found	No se crea préstamo; disponibilidad no cambia.
Libro inexistente	404 Not Found	No se crea préstamo.
Consulta de préstamos	200 OK	Devuelve el historial.
Datos inválidos	400 Bad Request	La validación rechaza identificadores faltantes.
Segundo préstamo activo	409 Conflict	Se conserva un solo préstamo activo para el libro.
Dos solicitudes concurrentes	201 y 409	Un solo registro; se confirma disponibilidad false.

La evidencia endpoints-local.txt registra respuestas reales locales: POST válido 201; libro no disponible 409; estudiante inexistente 404; libro inexistente 404; GET de préstamos 200 y cuerpo inválido 400.

Actividad 7. Revisión y defensa

¿Qué responsabilidades cambiaron del modelo inicial al modelo por capas?

GestorPrestamos deja de administrar directamente las listas y coordinar todas las operaciones. PrestamoController recibe HTTP; PrestamoService coordina el caso de uso; los repositorios consultan y persisten; Estudiante, Libro y Prestamo permanecen como entidades del dominio.

¿Qué patrón GRASP se reconoce en la coordinación? ¿Por qué Controller GRASP no equivale necesariamente a @RestController?

Controller GRASP es el rol que recibe y coordina un evento del sistema. En esta implementación, PrestamoController cumple el rol porque recibe el evento HTTP y delega en PrestamoService. @RestController es una anotación de Spring que configura un controlador web; es un mecanismo tecnológico, no la definición conceptual del patrón.

¿Dónde se ubica la regla de disponibilidad y por qué?

PrestamoService coordina la validación de la regla R1 y de la existencia de un préstamo activo (R5). Libro conoce su estado mediante isDisponible() y lo modifica con marcarComoPrestado(); por ello se reconoce Information Expert para el estado del libro.

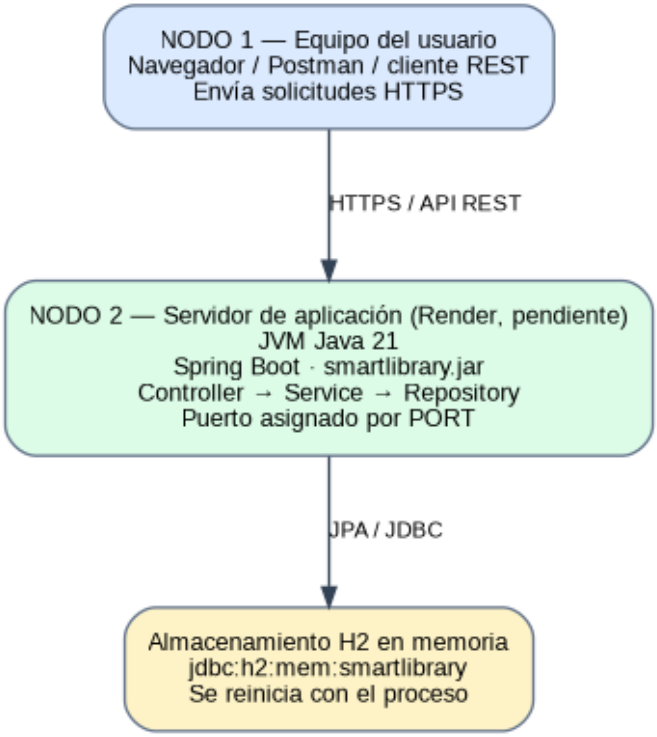
¿Qué relaciones UML se conservaron y cuáles cambiaron?

Se conservan las asociaciones históricas de Prestamo con Estudiante y Libro. En la implementación son referencias ManyToOne obligatorias desde Prestamo. Se retira GestorPrestamos y sus asociaciones de administración de listas. Se incorporan dependencias de Controller a Service y de Service a las interfaces Repository.

¿Cómo se demuestra la trazabilidad requisito → clase → operación → prueba?

R1 y R5 → PrestamoService, Libro y LibroRepository → registrar(), isDisponible(), findByIdForUpdate() y existsByLibroIdAndFechaDevolucionIsNull() → pruebas de 201, 409 y concurrencia. R2 → Prestamo → constructor con fecha y referencias → prueba 201. R3 → Libro.marcarComoPrestado() → verificación de disponible=false. R4 → PrestamoRequest y GlobalExceptionHandler → pruebas 400/404/409 sin registros parciales.

Diagrama de despliegue



El Dockerfile utiliza una etapa de construcción Maven 3.9.11 con Java 21 y una etapa de ejecución Eclipse Temurin 21 JRE. La aplicación escucha en 0.0.0.0 y utiliza el puerto PORT. render.yaml define el servicio smartlibrary-api y el healthCheckPath /api/prestamos.

Estado verificado al 12 de noviembre de 2026: el repositorio está publicado en GitHub, rama main, y el servicio smartlibrary-api se encuentra Live en Render. URL pública: **https://smartlibrary-api-2q8b.onrender.com**. Swagger UI está disponible en **/swagger-ui/index.html** y el documento OpenAPI en /v3/api-docs; ambas rutas fueron comprobadas en producción.

Explicación de decisiones de diseño

El diseño conserva Estudiante, Libro y Prestamo como entidades del dominio. Cada préstamo referencia obligatoriamente un estudiante y un libro; cada estudiante y libro puede participar en cero o muchos préstamos históricos. Son asociaciones, no composiciones, porque sus ciclos de vida son independientes.

GestorPrestamos concentraba la administración de colecciones, la coordinación del préstamo y las consultas. Para aumentar la cohesión y reducir el acoplamiento, esas responsabilidades se distribuyeron en tres capas: PrestamoController recibe solicitudes HTTP; PrestamoService coordina el caso de uso y las reglas; los repositorios encapsulan el acceso a datos. La dirección de dependencias es Controller → Service → Repository.

Se aplica Information Expert en Libro, que conoce y modifica su disponibilidad mediante isDisponible() y marcarComoPrestado(). PrestamoController cumple el rol Controller GRASP al recibir el evento del sistema y delegarlo; PrestamoService coordina la operación y construye Prestamo (Creator). PrestamoRequest y PrestamoResponse separan el contrato HTTP de las entidades JPA, mientras GlobalExceptionHandler centraliza los errores.

Para cumplir R5, PrestamoService obtiene el libro con bloqueo pesimista, verifica su disponibilidad y que no exista un préstamo activo. El registro es transaccional; si ocurre un error, se revierte la operación. Las nueve pruebas de integración cubren escenarios exitosos, negativos y concurrentes. La API está desplegada en Render y documentada con Swagger.